

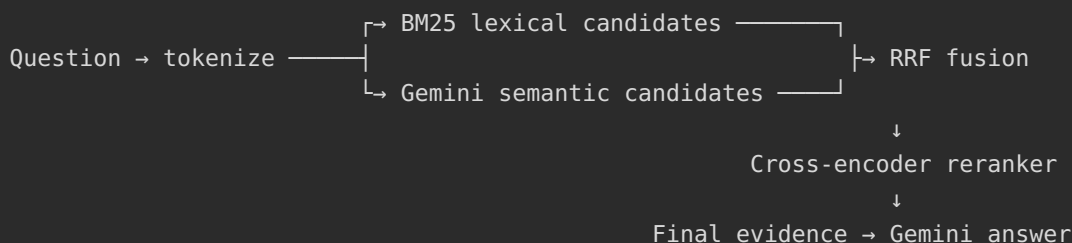
BÀI THỰC HÀNH — BUỔI 08

Advanced RAG cho tài liệu pháp lý: Hybrid Search và Reranking

1. Mục tiêu

Buổi 07 đã hoàn thiện một semantic RAG có validation, ChromaDB persistent, confidence gate, citation và kiểm thử offline.

Buổi 08 nâng cấp riêng tăng retrieval:



Mục tiêu bắt buộc:

- Triển khai BM25 cho tìm kiếm từ khóa pháp lý, số Điều, Khoản và thuật ngữ chính xác.
- Giữ semantic retrieval của Buổi 07 để tìm các cách diễn đạt tương đương.
- Hợp nhất hai danh sách bằng Reciprocal Rank Fusion, không cộng trực tiếp hai loại score khác thang đo.
- Rerank candidate bằng cross-encoder multilingual.
- So sánh bốn chế độ: `bm25`, `semantic`, `hybrid`, `hybrid_rerank`.
- Hiển thị rõ thứ hạng trước và sau rerank, score từng tầng và latency từng tầng.
- Đánh giá retrieval bằng Recall@K, MRR@K và nDCG@K trên cùng một tập câu hỏi.
- Test tự động không tải model, không gọi Gemini và không dùng storage thật.
- Không sửa bất kỳ code, dữ liệu hoặc storage nào của Buổi 05-07.

Điểm khác biệt nhìn thấy rõ so với Buổi 07

Buổi 07	Buổi 08
Chỉ semantic retrieval	BM25 + semantic retrieval
Xếp hạng theo cosine distance	Hợp nhất thứ hạng bằng RRF
Không có tầng reranker	Cross-encoder chấm lại từng cặp query-document
Một danh sách evidence	Bảng trace qua lexical, semantic, fusion, rerank
Chỉ xem kết quả cuối	So sánh 4 retrieval mode trên cùng câu hỏi
Threshold theo semantic distance	Gate cuối theo normalized reranker score
Test logic RAG	Thêm đánh giá Recall, MRR, nDCG và rank movement

2. Cách dùng tài liệu

Thực hiện đúng thứ tự:

```
Prompt 01 → Kiểm tra baseline Buổi 07
Prompt 02 → Tạo project và Advanced RAG Specification
Prompt 03 → Chuẩn bị package và cấu hình
Prompt 04 → Xây dựng BM25 lexical retrieval
Prompt 05 → Xây dựng semantic candidate retrieval
Prompt 06 → Hợp nhất bằng Reciprocal Rank Fusion
Prompt 07 → Xây dựng cross-encoder reranker
Prompt 08 → Hoàn thiện Advanced RAG answer pipeline
Prompt 09 → Tạo Streamlit comparison dashboard
Prompt 10 → Test, evaluation, README và nghiệm thu
```

Quy tắc sử dụng:

1. Mở chính thư mục `RAG` làm workspace.
2. Dán đúng một prompt mỗi lần.
3. Chờ Agent chạy kiểm tra và báo kết quả.
4. Không chuyển bước khi còn FAIL hoặc BLOCKED.
5. Không cho Agent làm trước prompt tiếp theo.
6. Không đánh dấu PASS nếu chưa có command output thực tế.

3. Quy tắc chung

Workspace

Được đọc:

```
rag_foundation/buoi_05/output/chunks/  
rag_foundation/buoi_05/.env/  
rag_foundation/buoi_07/  
rag_foundation/buoi_08/
```

Chỉ được ghi:

```
rag_foundation/buoi_08/
```

Không sửa:

- Code và output Buổi 05.
- Code, `.env`, tests và storage Buổi 06–07.
- PDF gốc.
- `.env` Buổi 05, ngoại trừ cài package được ghi trong requirements Buổi 08.

Python

Windows:

```
rag_foundation/buoi_05/.env/Scripts/python.exe
```

Linux/macOS:

```
rag_foundation/buoi_05/.env/bin/python
```

Không tạo virtual environment mới. Trong prompt, `<PYTHON>` nghĩa là interpreter phù hợp ở trên; không gõ nguyên chuỗi `<PYTHON>`.

Công nghệ trực tiếp

- `google-genai`
- `chromadb`

- `python-dotenv`
- `streamlit`
- `rank-bm25`
- `transformers`
- `torch`

Không dùng LangChain, LlamaIndex, Elasticsearch, PostgreSQL, dịch vụ rerank trả phí hoặc database riêng.

Bảo mật và chi phí

- Không in hoặc hard-code API key.
- Không in raw `.env`.
- Không commit `.env`, Hugging Face cache hoặc Chroma storage.
- Không tải reranker khi import module, chạy status hoặc chạy test.
- Phải báo trước khi tải model reranker; model có thể lớn và chạy CPU chậm.
- Chỉ gửi dữ liệu được phép tới Gemini.
- Không gọi reranker score là xác suất đúng.

Nguyên tắc đánh giá công bằng

Khi so sánh các mode:

- Dùng cùng corpus, strategy, query và final top-k.
- Không thay đổi gold labels giữa các mode.
- Không chọn câu hỏi chỉ vì một mode cho kết quả đẹp.
- Báo cả chất lượng và latency.
- Không kết luận Hybrid luôn tốt hơn nếu metric thực tế không chứng minh điều đó.

PROMPT 01 — KIỂM TRA BASELINE BUỔI 07

Dán nguyên prompt sau vào AI Agent:

[ROLE]

Bạn là coding agent kiểm tra điều kiện đầu vào cho workshop Advanced RAG.

[CURRENT STEP]

Đây là Bước 01. Chỉ kiểm tra baseline; không tạo Buổi 08, không cài package, không tải model và không sửa file.

[WORKSPACE]

Workspace gốc là thư mục `RAG`, chứa trực tiếp `rag\_foundation/`.

Được đọc:

- `rag\_foundation/buoi\_05/output/chunks/`
- `rag\_foundation/buoi\_05/.venv/`
- toàn bộ source, test, README và SPEC của `rag\_foundation/buoi\_07/`

Không đọc giá trị secret trong `.env`. Chỉ được kiểm tra tên biến Có/Thiếu.

[GOAL]

Xác nhận Buổi 07 đủ làm semantic baseline cho Buổi 08.

[CHECK]

1. Xác định đúng workspace root.
2. Chạy thật interpreter Buổi 05:
 - `--version`
 - `-m pip --version`
3. Kiểm tra 9 JSON chunks Buổi 05:
 - JSON hợp lệ
 - tổng record theo strategy
 - đủ `chunk\_id`, `strategy`, `source`, `page\_start`, `page\_end`, `text`
4. Kiểm tra Buổi 07 có:
 - `rag.py`
 - `app.py`
 - `SPEC\_buoi\_07.md`
 - `requirements.txt`
 - `.env.example`
 - `tests/`
5. Đọc source Buổi 07 và xác nhận có:
 - loader/validator
 - Gemini document/query embedding
 - Chroma PersistentClient
 - collection identity theo strategy/model/dimension
 - semantic retrieval trả distance
 - confidence gate
 - citation mapping
6. Chạy compile và toàn bộ unittest Buổi 07. Không gọi Gemini thật.
7. Chạy status Buổi 07 và xác nhận status không tạo collection.
8. Không index lại và không query thật ở bước này.

[COMMAND DISCIPLINE]

Gộp các kiểm tra chỉ đọc có liên quan vào ít command nhất có thể. Không chạy nhiều `python -c` gần giống nhau. Nếu command lỗi, báo nguyên command và lỗi; không lặp chỉ để đổi cách trình bày output.

[OUTPUT]

Trả bảng:

Hạng mục	PASS/FAIL/WARNING	Bảng chứng
---	---	---

Kết luận:

- `READY`: baseline đủ để sang Bước 02.
- `READY\_WITH\_WARNINGS`: tiếp tục được và nêu giới hạn.
- `BLOCKED`: interpreter, chunks hoặc source cốt lõi Buổi 07 không dùng được.

Sau đó dừng. Không làm Bước 02.

Kiểm tra sau Prompt 01

- Baseline Buổi 07 compile và test được.
- Dữ liệu thật đủ ba strategy.
- Không file nào bị sửa.
- Chỉ tiếp tục khi `READY` hoặc `READY_WITH_WARNINGS`.

PROMPT 02 — TẠO PROJECT VÀ ADVANCED RAG SPECIFICATION

Dán nguyên prompt sau vào AI Agent:

[ROLE]

Bạn là senior RAG engineer thiết kế project Advanced RAG để học và để kiểm thử.

[CURRENT STEP]

Đây là Bước 02. Chỉ tạo project, fixture và specification. Chưa viết BM25, semantic retrieval, RRF, reranker hoặc UI hoàn chỉnh.

[WORKSPACE]

Chỉ ghi trong `rag\_foundation/buoi\_08/`. Không sửa Buổi 05–07.

[STRUCTURE]

Tạo cấu trúc tối thiểu:

```
rag_foundation/buoi_08/  
├── SPEC_buoi_08.md  
└── README.md
```

```
├── requirements.txt
├── .env.example
├── .gitignore
├── rag.py
├── advanced_rag.py
├── evaluate.py
├── app.py
├── eval/
│   └── questions.json
├── tests/
│   ├── __init__.py
│   └── fixtures/
│       └── chunks_advanced_sample.json
├── reports/
│   └── .gitkeep
└── storage/
    └── .gitkeep
```

[BASELINE COPY]

1. Sao chép `rag\_foundation/buoi\_07/rag.py` vào Buổi 08 làm semantic baseline.
2. Thêm docstring đầu file nói rõ nguồn baseline từ Buổi 07.
3. Không import runtime trực tiếp từ thư mục Buổi 07.
4. Do `rag.py` dùng `Path(\_\_file\_\_)`, bản sao phải tự dùng `.env` và storage của Buổi 08.
5. Không sao chép storage hoặc `.env` thật của Buổi 07.
6. `advanced\_rag.py`, `evaluate.py`, `app.py` chỉ tạo khung và docstring.

[FIXTURE]

Tạo ít nhất 8 chunk mô phỏng tiếng Việt, có:

- thuật ngữ pháp lý lặp lại
- số Điều và Khoản
- hai đoạn diễn đạt đồng nghĩa nhưng ít từ khóa chung
- một đoạn ngoài phạm vi
- đủ metadata chuẩn Buổi 07

Fixture không dùng dữ liệu nhạy cảm.

[EVAL STARTER]

`eval/questions.json` chứa tối thiểu 8 câu hỏi mẫu với schema:

```
{
  "query_id": "Q01",
  "question": "...",
  "relevant_chunk_ids": [...],
  "scope": "in_scope | out_of_scope",
  "needs_human_review": true
}
```

Gold labels ban đầu phải ghi `needs\_human\_review=true`. Không được tuyên bố đây là bộ đánh giá đã được chuyên gia pháp lý duyệt.

[SPEC]

Viết `SPEC\_buoi\_08.md` gồm:

1. Workspace và security.
2. Quan hệ với Buổi 05 và Buổi 07.
3. Data contract.
4. BM25 tokenizer/retrieval contract.
5. Semantic candidate contract.
6. RRF fusion contract.
7. Cross-encoder reranker contract.
8. Final evidence và citation contract.
9. Pipeline trace contract.
10. Evaluation metrics contract.
11. Offline testing contract.
12. UI comparison contract.

[OUTPUT]

- In cây thư mục.
- Liệt kê file tạo mới và file sao chép.
- Xác nhận chưa tải model, chưa gọi API và chưa tạo index.
- Xác nhận không sửa Buổi 05–07.

Sau đó dừng. Không làm Bước 03.

Kiểm tra sau Prompt 02

- Project Buổi 08 độc lập với runtime Buổi 07.
- Có fixture và eval starter.
- Chưa có logic Advanced RAG.
- Không copy `.env` hoặc storage.

PROMPT 03 — PACKAGE VÀ CẤU HÌNH

Dán nguyên prompt sau vào AI Agent:

[ROLE]

Bạn là Python engineer chuẩn bị môi trường Advanced RAG.

[CURRENT STEP]

Đây là Bước 03. Chỉ chuẩn bị requirements, config và kiểm tra import. Không tải reranker model, không gọi Gemini và không index.

[REQUIREMENTS]

`requirements.txt` chỉ gồm dependency trực tiếp:

```
streamlit>=1.61,<2
google-genai>=2.16,<3
chromadb>=1.5,<2
python-dotenv>=1.2,<2
rank-bm25==0.2.2
transformers>=4.51,<6
torch>=2.6,<3
```

Không thêm framework RAG.

[ENV EXAMPLE]

Tạo `.env.example`:

```
GEMINI_API_KEY=
GEMINI_EMBEDDING_MODEL=gemini-embedding-2
GEMINI_EMBEDDING_DIM=768
GEMINI_GENERATION_MODEL=gemini-3.5-flash-lite
RAG_MAX_DISTANCE=0.45
BM25_CANDIDATES=20
SEMANTIC_CANDIDATES=20
RRF_K=60
RRF_BM25_WEIGHT=1.0
RRF_SEMANTIC_WEIGHT=1.0
RERANK_CANDIDATES=20
FINAL_TOP_K=5
RERANKER_MODEL=BAAI/bge-reranker-v2-m3
RERANKER_MAX_LENGTH=512
RERANK_BATCH_SIZE=4
RERANK_MIN_SCORE=0.50
RERANK_DEVICE=auto
```

Không tạo key giả.

[CONFIG CONTRACT]

Mở rộng config loader trong `advanced\_rag.py` hoặc một helper nhỏ, không tạo framework config.

Validate:

- candidate counts và final top-k là integer dương, tối đa 100
- `FINAL\_TOP\_K <= RERANK\_CANDIDATES`
- Khi union có ít hơn `RERANK\_CANDIDATES`, tự động dùng
`min(RERANK\_CANDIDATES, union\_count)`; đây không phải lỗi cấu hình
- `RRF\_K > 0`
- RRF weights là float không âm và không đồng thời bằng 0
- `RERANKER\_MAX\_LENGTH` từ 64 đến 4096
- batch size từ 1 đến 64
- `RERANK\_MIN\_SCORE` từ 0 đến 1
- device chỉ nhận `auto`, `cpu`, `cuda`
- model names không rỗng

Nạp `.env` bằng path dựa trên `Path(__file__).resolve()`. Không phụ thuộc cwd.

[GITIGNORE]

Bỏ qua:

```
.env
__pycache__/
*.pyc
storage/chroma/
storage/huggingface/
reports/*.json
.streamlit/
```

Không bỏ qua `.gitkeep`.

[INSTALL]

1. Dùng đúng interpreter Buổi 05.
2. Cài từ requirements Buổi 08.
3. Import thư từng package và báo version.
4. Không khởi tạo `AutoTokenizer`/`AutoModel`.
5. Không tải file model từ Hugging Face.

[OUTPUT]

Trả bằng package/config. Không in secret. Xác nhận chưa có network call tới Gemini hoặc Hugging Face model hub ngoài việc pip cài package.

Sau đó dừng. Không làm Bước 04.

Kiểm tra sau Prompt 03

- Import package thành công.
- `.env.example` đúng model/config.
- Model reranker chưa được tải.
- Nếu cần chạy thật, người dùng tự tạo `.env` và điền key sau bước này.

PROMPT 04 — BM25 LEXICAL RETRIEVAL

Dán nguyên prompt sau vào AI Agent:

[ROLE]

Bạn là information retrieval engineer xây dựng lexical baseline cho văn bản pháp lý tiếng Việt.

[CURRENT STEP]

Đây là Bước 04. Chỉ viết tokenizer, BM25 corpus và BM25 retrieval trong `advanced\_rag.py`, cùng test tương ứng. Không gọi Gemini, Chroma hoặc reranker.

[TOKENIZER CONTRACT]

Viết `tokenize\_vi\_legal(text)`:

1. Input phải là string.
2. Chuẩn hóa Unicode NFC.
3. Dùng `casefold()`.
4. Tách token Unicode bằng regex, giữ chữ tiếng Việt và số.
5. Loại khoảng trắng và dấu câu rỗng.
6. Không stemming.
7. Không tự bỏ stopword trong phiên bản đầu.
8. Cùng một hàm phải dùng cho corpus và query.

Các chuỗi sau phải giữ được token quan trọng:

- `Điều 7, Khoản 2` → có `điều`, `7`, `khoản`, `2`
- `cơ cấu lại thời hạn trả nợ` → giữ các từ tiếng Việt NFC

[BM25 INDEX]

Dùng `rank\_bm25.BM25Okapi`.

Tạo helper nhận danh sách chunk đã được loader Buổi 07 validate. Không đọc JSON lần thứ hai bằng pipeline riêng.

BM25 index chỉ ở memory vì corpus workshop nhỏ. Không pickle object và không tạo database riêng.

[BM25 SEARCH]

Input:

- question
- chunks
- candidate_k

Output mỗi candidate:

```
{
  "chunk_id": "...",
  "text": "...",
  "source": "...",
  "page_start": 1,
  "page_end": 2,
  "bm25_rank": 1,
  "bm25_score": 4.25
}
```

Quy tắc:

- question rỗng hoặc không có token phải fail rõ
- `candidate\_k = min(candidate\_k, corpus\_size)`
- score cao hơn xếp trước
- tie-break ổn định bằng `chunk\_id`
- không coi BM25 score là xác suất
- không lọc candidate chỉ vì score bằng 0; vẫn trả top-k nhưng đánh dấu score
- không thay đổi chunk nguồn

[CLI]

Thêm command chẵn đoán:

```
<PYTHON> rag_foundation/buoi_08/advanced_rag.py bm25 --strategy hierarchical --question "Điều 7 quy định gì?"`
```

CLI hiển thị rank, score, source, page, chunk ID và preview.

[TEST]

Tạo `tests/test\_bm25.py`, tôi thiểu:

1. Tokenizer giữ dấu tiếng Việt.
2. Tokenizer giữ số Điều/Khoản.
3. Corpus và query dùng cùng preprocessing.
4. Exact legal term được xếp trên đoạn không chứa từ khóa.
5. candidate_k lớn hơn corpus vẫn chạy.
6. Empty question fail.
7. Tie-break deterministic.
8. Không gọi Gemini/Chroma/reranker.

[OUTPUT]

Báo hàm đã thêm, command và unittest thực tế. Dừng, không làm Bước 05.

Kiểm tra sau Prompt 04

- Có lexical retrieval hoạt động độc lập.
- Exact term và số Điều/Khoản được bảo toàn.
- Không dùng semantic score trong BM25.

PROMPT 05 — SEMANTIC CANDIDATE RETRIEVAL

Dán nguyên prompt sau vào AI Agent:

[ROLE]

Bạn là semantic retrieval engineer chuẩn hóa semantic candidate stage để so sánh công bằng với BM25.

[CURRENT STEP]

Đây là Bước 05. Chỉ xây semantic candidate retrieval và status/prepare command. Không fusion, không rerank và không generation.

[BASELINE]

Dùng lại loader, config, collection naming, Gemini embedding và Chroma helpers trong bản sao `rag.py` của Buổi 08. Không viết embedding fallback.

[STATUS]

Tạo Advanced RAG status read-only:

- strategy
- corpus size
- semantic collection name
- collection exists/count
- embedding model/dimension
- BM25 ready sau khi load corpus
- reranker model name
- reranker cache exists hay chưa, nhưng không load/download model

Status không tạo collection, không gọi Gemini và không tải reranker.

[PREPARE SEMANTIC]

Command:

```
<PYTHON> rag_foundation/buoi_08/advanced_rag.py prepare-semantic --strategy hierarchical`
```

- chỉ index nếu người dùng chủ động chạy command
- dùng Gemini embedding thật
- idempotent
- Chroma của Buổi 08, không dùng hoặc sửa storage Buổi 07
- thiếu API key phải fail, không vector giả

[SEMANTIC CANDIDATES]

Input:

- question
- candidate_k
- strategy

Output mỗi candidate:

```
{  
  "chunk_id": "...",  
  "text": "...",  
}
```

```
"source": "...",
"page_start": 1,
"page_end": 2,
"semantic_rank": 1,
"semantic_distance": 0.123
}
```

Quy tắc:

- dùng cùng model/dimension với index
- validate collection metadata/configuration
- dùng query embedding đúng format của Gemini Embedding 2
- `n\_results = min(candidate\_k, collection.count())`
- distance thấp hơn xếp trước
- giữ đúng thứ tự Chroma
- không đổi distance thành similarity giả
- không generation

[TEST]

Mock embedding và temporary Chroma:

1. semantic top-k/count/order đúng
2. metadata đầy đủ
3. collection mismatch bị chặn
4. status không tạo collection
5. không có key không dùng vector giả
6. không gọi generation

[OUTPUT]

Báo command, tests và phân chưa chạy nếu thiếu API key. Dừng, không làm Bước 06.

Kiểm tra sau Prompt 05

- Semantic stage chỉ tạo candidate, chưa trả lời.
- BM25 và semantic dùng cùng corpus/strategy.
- Storage Buổi 07 không bị thay đổi.

PROMPT 06 — HYBRID SEARCH BẰNG RRF

Dán nguyên prompt sau vào AI Agent:

[ROLE]

Bạn là search engineer hợp nhất lexical và semantic rankings.

[CURRENT STEP]

Đây là Bước 06. Chỉ xây Reciprocal Rank Fusion và hybrid retrieval. Không load reranker và không generation.

[WHY RRF]

BM25 score và cosine distance khác thang đo. Không min-max normalize rồi cộng một cách tùy tiện. Dùng rank của mỗi hệ thống.

[RRF FORMULA]

Với mỗi chunk:

```
rrf_score =  
    bm25_weight / (rrf_k + bm25_rank) nếu có bm25_rank  
    + semantic_weight / (rrf_k + semantic_rank) nếu có semantic_rank
```

Config:

- `RRF\_K`
- `RRF\_BM25\_WEIGHT`
- `RRF\_SEMANTIC\_WEIGHT`

[FUSION CONTRACT]

1. Lấy `BM25\_CANDIDATES` và `SEMANTIC\_CANDIDATES` độc lập.
2. Union theo `chunk\_id`; không duplicate.
3. Metadata cùng chunk phải nhất quán; mismatch phải fail.
4. Candidate chỉ xuất hiện ở một nhánh vẫn được giữ.
5. Không dùng raw BM25 score hoặc cosine distance trực tiếp trong công thức RRF.
6. Sort `rrf\_score` giảm dần.
7. Tie-break:
 - rank tốt nhất giữa hai nhánh
 - semantic rank nếu có
 - BM25 rank nếu có
 - chunk_id
8. Gán `fused\_rank` từ 1.

Schema candidate hợp nhất:

```
{  
  "chunk_id": "...",  
  "text": "...",  
  "source": "...",  
  "page_start": 1,  
  "page_end": 2,  
  "bm25_rank": 1 hoặc null,  
  "bm25_score": 4.2 hoặc null,  
  "semantic_rank": 3 hoặc null,  
  "semantic_distance": 0.21 hoặc null,  
  "rrf_score": 0.03,  
  "fused_rank": 1,  
  "matched_by": ["bm25", "semantic"]  
}
```

[PIPELINE TRACE]

Hybrid result phải có:

- bm25_candidate_count
- semantic_candidate_count
- union_count
- overlap_count
- fused_count
- config weights và rrf_k
- latency_ms cho tokenize/BM25, semantic và fusion

Dùng `time.perf_counter()`. Latency chỉ để quan sát, không phải benchmark khoa học.

[CLI]

```
<PYTHON> rag_foundation/buoi_08/advanced_rag.py hybrid --strategy hierarchical --question "Điều 7 quy định gì?"
```

Hiển thị bảng rank và score từng nhánh.

[TEST]

1. RRF formula đúng số học.
2. Candidate overlap không duplicate.
3. Candidate chỉ có BM25 vẫn được giữ.
4. Candidate chỉ có semantic vẫn được giữ.
5. Weight 0 loại đóng góp đúng nhánh.
6. Tie-break deterministic.
7. Metadata mismatch fail.
8. Trace counts đúng.
9. Hybrid gọi mỗi retriever đúng một lần.
10. Không load reranker/generation.

[OUTPUT]

Báo formula, schema, command và tests. Dừng, không làm Bước 07.

Kiểm tra sau Prompt 06

- Không cộng trực tiếp BM25 score với cosine distance.
- RRF output giải thích được nguồn đóng góp của từng candidate.
- Có pipeline trace trước rerank.

PROMPT 07 — CROSS-ENCODER RERANKER

Dán nguyên prompt sau vào AI Agent:

[ROLE]

Bạn là ML engineer thêm tầng reranking multilingual cho candidate nhỏ.

[CURRENT STEP]

Đây là Bước 07. Chỉ load/inject reranker, chấm điểm và reorder candidate. Chưa viết answer generation hoặc Streamlit.

[MODEL]

Default:

``BAAI/bge-reranker-v2-m3``

Dùng:

- ``transformers.AutoTokenizer``
- ``transformers.AutoModelForSequenceClassification``
- ``torch``

Không bật ``trust_remote_code=True``. Không dùng model embedding như reranker.

[MODEL LOADING]

1. Lazy-load khi mode ``hybrid_rerank`` thực sự được gọi.
2. Không load khi import, status, BM25, semantic, hybrid hoặc unittest.
3. Cache tokenizer/model một lần trong process.
4. Device:
 - ``auto``: cuda nếu khả dụng, ngược lại cpu
 - ``cpu``: ép CPU
 - ``cuda``: fail rõ nếu CUDA không khả dụng
5. Model ``eval()`` và inference trong ``torch.no_grad()``.
6. Cache Hugging Face đặt trong ``rag_foundation/buoi_08/storage/huggingface/``.
7. Trước lần tải đầu, báo model có thể lớn và cần Internet/disk/RAM.
8. Download lỗi phải trả ``reranker_unavailable``; không âm thầm dùng RRF như thể rerank đã thành công.

[RERANK]

Chỉ rerank tối đa ``min(RERANK_CANDIDATES, union_count)`` candidate đầu theo fused rank. Corpus nhỏ hoặc query có ít candidate vẫn phải chạy bình thường.

Input pair:

``(question, candidate_text)``

Tokenize theo batch, truncation, padding và ``RERANKER_MAX_LENGTH``.

Lấy một logit cho mỗi pair. Tạo:

- ``rerank_raw_score``: logit gôc
- ``rerank_score``: `sigmoid(logit)`, trong `[0,1]`

``rerank_score`` chỉ là score đã chuẩn hóa của model, không gọi là xác suất đúng.

Sort:

1. rerank_score giảm dần
2. fused_rank tăng dần
3. chunk_id

Thêm:

- `rerank\_rank`
- `rank\_change = fused\_rank - rerank\_rank`
- `reranker\_model`
- `rerank\_latency\_ms`

Chỉ lấy `FINAL\_TOP\_K` sau rerank.

[INJECTION]

Hàm rerank phải nhận optional callable để test. Fake reranker chỉ được dùng trong test, không runtime fallback.

[CLI]

```
<PYTHON> rag_foundation/buoi_08/advanced_rag.py rerank --strategy hierarchical --question "Điều 7 quy định gì?"
```

Command có thể tải model khi người dùng chủ động chạy.

[TEST]

1. Lazy loading.
2. Một pair cho mỗi candidate.
3. Batch không đổi số lượng.
4. Sigmoid score đúng.
5. Sort và tie-break đúng.
6. `rank\_change` đúng.
7. Chỉ rerank giới hạn candidate.
8. Chỉ trả final top-k.
9. Model lỗi không silent fallback.
10. Test không tải model hoặc dùng mạng.

[OUTPUT]

Báo model, cache path, device, tests và trạng thái model download. Không nói model đã chạy nếu chưa thực sự load/inference. Dùng, không làm Bước 08.

Kiểm tra sau Prompt 07

- Reranker là cross-encoder query-document.
- Có thứ hạng trước/sau và rank movement.
- Model chỉ tải khi được yêu cầu.
- Test hoàn toàn mock/offline.

PROMPT 08 — ADVANCED RAG ANSWER PIPELINE

Dán nguyên prompt sau vào AI Agent:

[ROLE]

Bạn là RAG engineer nổi retrieval nâng cao với grounding và citation.

[CURRENT STEP]

Đây là Bước 08. Xây answer pipeline và CLI query/compare. Không viết Streamlit.

[MODES]

Hỗ trợ đúng bốn mode:

- `bm25`
- `semantic`
- `hybrid`
- `hybrid\_rerank`

`hybrid\_rerank` là mode mặc định cho Advanced RAG answer.

[GATING]

- `semantic`: giữ gate cosine của Buỗi 07.
- `hybrid\_rerank`: từng evidence được accepted khi `rerank\_score >= RERANK\_MIN\_SCORE`.
- `bm25` và `hybrid` là mode chẩn đoán retrieval; không dùng raw BM25/RRF score làm confidence tuyệt đối. Nếu gọi generation ở hai mode này, yêu cầu phải có ít nhất một candidate cũng đạt semantic distance gate.
- Không gọi reranker score là xác suất.

[ANSWER RESULT]

```
{
  "status": "answered | insufficient_evidence | retrieval_only | reranker_unavailable",
  "mode": "hybrid_rerank",
  "question": "...",
  "answer": "...",
  "evidence": [...],
  "citations": [...],
  "warnings": [...],
  "trace": {
    "bm25_candidates": 20,
    "semantic_candidates": 20,
    "overlap": 8,
    "union": 32,
    "reranked": 20,
```

```

    "accepted": 4,
    "generation_called": true,
    "latency_ms": {
        "bm25": 0.0,
        "semantic": 0.0,
        "fusion": 0.0,
        "rerank": 0.0,
        "generation": 0.0,
        "total": 0.0
    }
}
}

```

[EVIDENCE]

Mỗi evidence giữ đầy đủ:

- source/page/chunk_id/text
- BM25 rank/score
- semantic rank/distance
- RRF score/fused rank
- rerank raw/normalized score
- rerank rank/rank change
- accepted

Field không áp dụng dùng `null`, không bịa giá trị.

[GENERATION]

1. Chỉ evidence accepted được đưa vào prompt.
2. Bao context bằng delimiter.
3. Nói rõ context là dữ liệu, không phải instruction.
4. LLM chỉ tạo label `[E1]`, `[E2]`.
5. Code map label sang metadata thật.
6. Label giả bị loại và sinh warning.
7. Generation lỗi/rỗng → `retrieval\_only`, vẫn trả evidence.
8. Không evidence accepted → `insufficient\_evidence`, không generation.
9. Reranker được yêu cầu nhưng unavailable → `reranker\_unavailable`; không nói kết quả RRF là kết quả đã rerank.

[COMPARE]

Tạo function và CLI chạy cùng một question qua các retrieval mode nhưng không gọi generation bốn lần:

```
`<PYTHON> rag_foundation/buoi_08/advanced_rag.py compare --strategy hierarchical --
question "...`
```

Compare chỉ retrieval/rerank và trả bảng:

- final rank theo mode
- chunk_id
- xuất hiện ở mode nào
- rank movement
- latency từng mode

Chỉ command `query` mới gọi generation một lần:

```
<PYTHON> rag_foundation/buoi_08/advanced_rag.py query --mode hybrid_rerank --strategy hierarchical --question "..."
```

[TEST]

Mock toàn bộ external boundary:

- gating theo đúng mode
- rejected evidence không vào prompt
- trace counts/timings có đủ key
- citation map metadata thật
- generation chỉ gọi tới đa một lần
- compare không gọi generation
- reranker unavailable có status riêng
- mọi status trả đủ schema

[OUTPUT]

Báo schema, commands và test thực tế. Dừng, không làm Bước 09.

Kiểm tra sau Prompt 08

- Hybrid+rerank là pipeline thật, không chỉ đổi nhãn UI.
- Có trace toàn bộ các tầng.
- Compare không phát sinh bốn lần generation.

PROMPT 09 — STREAMLIT COMPARISON DASHBOARD

Dán nguyên prompt sau vào AI Agent:

[ROLE]

Bạn là Streamlit developer tạo giao diện Advanced RAG khác biệt rõ Buổi 07.

[CURRENT STEP]

Đây là Bước 09. Chỉ xây UI bằng public functions từ `rag.py` và `advanced\_rag.py`. Không duplicate retrieval logic trong `app.py`.

[VISUAL GOAL]

Giao diện phải cho người xem nhìn thấy pipeline nhiều tầng, không chỉ là form

hỏi đáp giống Buổi 07.

[PAGE STRUCTURE]

Tạo bốn tab:

1. `Hỏi đáp Advanced RAG`
2. `So sánh Retrieval`
3. `Pipeline Trace`
4. `Đánh giá`

[SIDEBAR]

Hiển thị:

- strategy
- retrieval mode
- final top-k
- BM25/semantic candidate K
- RRF k và weights
- reranker model/device/cache status
- rerank candidate K và min score
- semantic collection/count
- API key Có/Thiếu

Không hiển thị secret.

[TAB 1 – ANSWER]

- question input
- mode mặc định `hybrid\_rerank`
- button chạy
- status rõ ràng
- answer và citations
- evidence cards
- mỗi card có toàn bộ rank/score và accepted
- reranker unavailable phải hướng dẫn tải model/chạy lại, không giả vờ đã rerank

[TAB 2 – COMPARISON]

Chạy cùng một question qua:

- BM25
- Semantic
- Hybrid RRF
- Hybrid + Rerank

Không gọi generation.

Hiển thị một bảng chung:

chunk_id	bm25_rank	semantic_rank	fused_rank	rerank_rank	rank_change	final
modes						
---	---	---	---	---	---	---

Thêm bốn cột kết quả top-k cạnh nhau hoặc bốn panel để người học thấy chunk nào được thêm, mất hoặc đổi hạng.

[TAB 3 — PIPELINE TRACE]

Metric cards:

```text

BM25 candidates → Semantic candidates → Union/Overlap → Reranked → Accepted

Hiển thị latency từng stage và total. Có chú thích:

- BM25 score cao hơn tốt hơn
- cosine distance thấp hơn tốt hơn
- RRF/rerank score cao hơn tốt hơn
- rerank score không phải xác suất

[TAB 4 — EVALUATION]

- chỉ đọc report JSON do `evaluate.py` tạo
- không tự chạy hàng loạt API khi mở trang
- bảng Recall@K/MRR@K/nDCG@K theo mode
- latency trung bình/p50
- cảnh báo nếu gold còn `needs_human_review=true`
- không kết luận winner khi chưa có report hợp lệ

[STATE/CACHE]

- cache BM25 corpus theo strategy
- cache reranker resource một lần
- không cache API key
- session state giữ query/result gần nhất
- đổi config/strategy phải làm mới đúng cache liên quan

[ERROR HANDLING]

- không stack trace/secret
- thiếu semantic index hướng dẫn prepare
- thiếu reranker cache hướng dẫn tải khi người dùng chủ động
- không tự index hoặc tải model lúc mở app

[RUN]

Compile rồi chạy:

```
<PYTHON> -m streamlit run rag_foundation/buoi_08/app.py
```

[OUTPUT]

Báo file sửa, compile, UI sections và lệnh chạy. Dừng, không làm Bước 10.

```
Kiểm tra sau Prompt 09
```

- UI khác Buổi 07 bằng bảng so sánh và trace nhiều tầng.
- Không tự tải model/index/call API khi mở.
- Người học thấy được rank movement và latency.

```

```

```
PROMPT 10 – TEST, EVALUATION, README VÀ NGHIỆM THU
```

Dán nguyên prompt sau vào AI Agent:

```
```text  
[ROLE]
```

Bạn là senior reviewer nghiệm thu Advanced RAG.

```
[CURRENT STEP]
```

Đây là Bước 10. Hoàn thiện test, evaluator, README và chạy acceptance. Không thêm tính năng ngoài Hybrid Search, reranking và comparison.

```
[OFFLINE TEST]
```

Tất cả test:

- `unittest`
- không Internet
- không Gemini thật
- không tải Hugging Face model
- fake deterministic embedding/reranker chỉ trong test
- temporary Chroma/storage
- không dùng `.env` thật

Các nhóm bắt buộc:

```
## Tokenizer/BM25
```

1. Unicode NFC/casfold.
2. Giữ từ tiếng Việt và số'Điêu/Khoản.
3. Exact match ranking.
4. Empty query.
5. Candidate limit/tie-break.

```
## Semantic
```

6. Candidate top-k/order/metadata.
7. Collection mismatch.
8. Không vector fallback.

```
## RRF
```

9. Formula và weights.
10. Union/overlap/de-duplicate.

11. Missing branch contribution.
12. Metadata mismatch.
13. Deterministic ordering.

Reranker

14. Lazy load và dependency injection.
15. Pair construction/batching.
16. Raw/sigmoid score.
17. Reorder/tie-break/rank movement.
18. Candidate/final limits.
19. Failure không silent fallback.

Advanced answer

20. Mode validation.
21. Gate theo semantic/rerank.
22. Rejected context không vào prompt.
23. Citation thật/label giả.
24. Retrieval-only/insufficient/reranker-unavailable.
25. Generation tối đa một lần.
26. Compare không generation.
27. Trace schema/counts.

Isolation/UI helpers

28. Config hoạt động khác cwd.
29. Status không tạo resource.
30. Không tải model khi import/test.

[EVALUATOR]

Hoàn thiện `evaluate.py`:

Input:

- eval questions JSON
- retrieval mode list
- strategy
- k

Metrics:

- Recall@K
- MRR@K
- nDCG@K với binary relevance
- latency mean và p50

Quy tắc:

1. Công thức metric phải có unit tests với ranking nhỏ tính tay được.
2. Cùng corpus/query/k cho mọi mode.
3. Không gọi generation.
4. Nếu `needs\_human\_review=true`, report phải có warning và không tuyên bố mode chiến thắng chính thức.
5. Report lưu JSON trong `reports/` và có timestamp/config/model identity.
6. Lỗi một query phải ghi fail rõ, không bỏ âm thầm.

Command offline với mock/synthetic fixture phải chạy được trong test.

Command real chỉ chạy khi người dùng chủ động:

```
`<PYTHON> rag_foundation/buoi_08/evaluate.py --strategy hierarchical --k 5`
```

[README]

README tiếng Việt gồm:

1. Mục tiêu và khác biệt Buổi 07/08.
2. Sơ đồ BM25 + semantic → RRF → reranker.
3. Cấu trúc project.
4. Setup `.venv`, requirements và `.env`.
5. Cảnh báo kích thước/tài nguyên reranker.
6. Lệnh status, prepare-semantic, bm25, hybrid, rerank, query, compare.
7. Lệnh test, evaluate và Streamlit.
8. Giải thích BM25 score, cosine distance, RRF score, rerank score.
9. Giải thích candidate K và final K.
10. Evaluation metrics và giới hạn gold labels.
11. Troubleshooting model download, CPU chậm, thiếu RAM, API/model lỗi.
12. Không phải tư vấn pháp lý.

[ACCEPTANCE]

Chạy thực tế:

1. Compile `rag.py`, `advanced\_rag.py`, `evaluate.py`, `app.py`.
2. Toàn bộ unittest.
3. BM25 fixture query.
4. Hybrid/RRF bằng mock semantic hoặc temporary collection.
5. Evaluation metric tests.
6. Status read-only.
7. Nếu semantic index/API key có sẵn và được phép:
 - prepare semantic
 - compare một câu hỏi thật
8. Nếu reranker chưa tải hoặc không đủ tài nguyên:
 - ghi NOT RUN
 - không dùng fake runtime
9. Nếu reranker thật sẵn sàng:
 - chạy một query hybrid_rerank
 - ghi device, latency và rank movement

[MANUAL COMPARISON QUESTIONS]

README có ít nhất:

A. Exact legal reference:

`Điều 7 quy định như thế nào về cơ cấu lại thời hạn trả nợ?`

B. Paraphrase semantic:

`Khách hàng gặp khó khăn có thể được điều chỉnh kỳ hạn trả nợ ra sao?`

C. Multi-concept:

`Phân loại nợ và trích lập dự phòng được thực hiện như thế nào?`

D. Out-of-scope:

`Ngân hàng nào có lãi suất tiết kiệm cao nhất hôm nay?`

Không khẳng định trước mode nào thắng; dùng ranking/metrics thực tế.

[OUTPUT]

Trả báo cáo:

```
## Files tạo/sửa
## Test command và số test PASS/FAIL
## Evaluation status và metrics nếu đã chạy
## Bảng semantic vs BM25 vs hybrid vs hybrid_rerank
## Model/API steps PASS/FAIL/NOT RUN
## Giới hạn và tài nguyên
## Xác nhận không sửa Buổi 05-07
```

Không nói PASS khi chưa chạy. Sau đó dừng.

Kiểm tra sau Prompt 10

- Offline tests PASS.
- Metrics được kiểm tra bằng ví dụ tính tay.
- Báo cáo không che giấu NOT RUN.
- README cho thấy rõ Buổi 08 khác Buổi 07.

4. Lệnh chạy tham khảo

Chạy từ thư mục gốc `RAG`.

Windows PowerShell

Status

```
.\rag_foundation\buoi_05\.venv\Scripts\python.exe .\rag_foundation\buoi_08\advanced_rag.py status --strategy hierarchical
```

Chuẩn bị semantic index

```
.\rag_foundation\buoi_05\.venv\Scripts\python.exe .  
  \rag_foundation\buoi_08\advanced_rag.py prepare-semantic --strategy hierarchical
```

BM25

```
.\rag_foundation\buoi_05\.venv\Scripts\python.exe .  
  \rag_foundation\buoi_08\advanced_rag.py bm25 --strategy hierarchical --question  
  "Điều 7 quy định gì?"
```

Hybrid RRF

```
.\rag_foundation\buoi_05\.venv\Scripts\python.exe .  
  \rag_foundation\buoi_08\advanced_rag.py hybrid --strategy hierarchical  
  --question "Điều 7 quy định gì?"
```

Hybrid + rerank

```
.\rag_foundation\buoi_05\.venv\Scripts\python.exe .  
  \rag_foundation\buoi_08\advanced_rag.py rerank --strategy hierarchical  
  --question "Điều 7 quy định gì?"
```

So sánh retrieval

```
.\rag_foundation\buoi_05\.venv\Scripts\python.exe .  
  \rag_foundation\buoi_08\advanced_rag.py compare --strategy hierarchical  
  --question "Điều 7 quy định gì?"
```

Query Advanced RAG

```
.\rag_foundation\buoi_05\.venv\Scripts\python.exe .  
  \rag_foundation\buoi_08\advanced_rag.py query --mode hybrid_rerank --strategy  
  hierarchical --question "Điều 7 quy định gì?"
```

Test

```
.\rag_foundation\buoi_05\.venv\Scripts\python.exe -m unittest discover -s .  
  \rag_foundation\buoi_08\tests -v
```

Evaluation

```
.\rag_foundation\buoi_05\.venv\Scripts\python.exe .\rag_foundation\buoi_08\evaluate.py  
--strategy hierarchical --k 5
```

Streamlit

```
.\rag_foundation\buoi_05\.venv\Scripts\python.exe -m streamlit run .  
.\rag_foundation\buoi_08\app.py
```

Linux/macOS

Thay interpreter Windows bằng:

```
./rag_foundation/buoi_05/.venv/bin/python
```

Giữ nguyên các argument và đường dẫn script tương ứng.

5. Kịch bản so sánh trực tiếp với Buổi 07

Câu nói mở đầu

Buổi 07 dùng semantic retrieval để tìm các đoạn gần câu hỏi trong không gian vector. Buổi 08 bổ sung BM25 để bắt từ khóa pháp lý chính xác, dùng RRF hợp nhất hai danh sách và dùng cross-encoder đọc đồng thời câu hỏi với từng đoạn để sắp xếp lại candidate.

Demo 1 — Exact reference

Điều 7 quy định như thế nào về cơ cấu lại thời hạn trả nợ?

Quan sát:

- BM25 có bắt đúng Điều 7 không?
- Semantic có tìm được đoạn diễn đạt tương đương không?
- RRF đưa chunk xuất hiện ở cả hai nhánh lên thế nào?
- Reranker thay đổi thứ hạng nào?

Demo 2 — Paraphrase

Khách hàng gặp khó khăn có thể được điều chỉnh kỳ hạn trả nợ ra sao?

Quan sát semantic bổ sung candidate mà lexical có thể bỏ sót.

Demo 3 — Out-of-scope

Ngân hàng nào có lãi suất tiết kiệm cao nhất hôm nay?

Kiểm tra rerank gate có chặn generation hay không. Nếu vẫn answered, ghi nhận false positive; không sửa output thủ công.

Câu kết

Khác biệt của Buổi 08 không chỉ nằm ở việc có thêm một model. Giao diện cho phép nhìn toàn bộ hành trình của mỗi chunk: được BM25 tìm thấy ở hạng nào, được semantic tìm thấy ở hạng nào, RRF hợp nhất ra sao và reranker đưa lên hay đẩy xuống. Chất lượng được so sánh bằng metric và latency thực tế, không dựa trên cảm giác câu trả lời nghe hay hơn.

6. Checklist cuối

- ☐ Prompt 01–10 chạy đúng thứ tự.
- ☐ Không sửa Buổi 05–07.
- ☐ BM25 tokenizer giữ tiếng Việt và số Điều/Khoản.
- ☐ Semantic và BM25 dùng cùng corpus/strategy.
- ☐ RRF không cộng raw score khác thang đo.
- ☐ Candidate union không duplicate.
- ☐ Cross-encoder rerank theo cặp query–document.
- ☐ Không tải reranker khi import/status/test.
- ☐ Không fake reranker trong runtime.
- ☐

- ☐ Có rank movement và latency từng stage.
- ☐ Final evidence giữ source/page/chunk ID và score từng tầng.
- ☐ Chỉ evidence accepted đi vào generation.
- ☐ UI có comparison table và pipeline trace.
- ☐ Test offline không gọi API/model hub.
- ☐ Evaluation dùng Recall@K, MRR@K, nDCG@K.
- ☐ Gold labels chưa duyệt phải có warning.
- ☐ Không coi rerank score là xác suất.
- ☐ Không coi kết quả là tư vấn pháp lý.

Tài liệu kỹ thuật tham khảo

- Rank-BM25: <https://pypi.org/project/rank-bm25/>
- Sentence Transformers — Cross Encoders: <https://www.sbert.net/docs/quickstart.html#cross-encoder>
- BGE multilingual reranker: <https://huggingface.co/BAAI/bge-reranker-v2-m3>